

Example:

GET /hackathons

POST /hackathons

PATCH /hackathons/123

DELETE /hackathons/123

Yahan 123 hackathon ki ID ho sakti hai.

API Versioning

Ek aur important rule:

/api/v1/...

Hum API ko version ke saath design karenge.

Future me agar API ka structure completely change karna pade:

/api/v1/...

/api/v2/...

Purane clients ko turant todna nahi padega.

Ek aur important correction ★

One API request is not always better than many API requests.

Dashboard ke liye ek combined API useful hai.

Lekin agar kisi page ko sirf notifications chahiye, to poora dashboard data bhejna wasteful hoga.

Isliye API design **page aur use-case ke according** hona chahiye.

Ab next: Authentication ka actual technical design 🔒

Abhi tak humne authentication ko business level par samjha tha.

Ab hum decide karenge:

Signup



Password securely store



Login



Authentication Token



Frontend stores token



Every protected API request



Backend verifies token



Access granted / denied

Yahan hum **password hashing, JWT/session, access token, refresh token aur role/permission checking** samjhenge.

Perfect bhai. 😊 Ab hum architecture ka ek **bahut important security chapter** start karte hain.

Chapter 31: Authentication Architecture 🔒

Authentication verifies identity. Authorization verifies permission.

Hum ye distinction pehle bhi samajh chuke hain. Ab dekhte hain technically ye kaise work karega.

1. Signup

User account banata hai:

Name

Email

Password



Frontend



Backend API



Validate Data



Hash Password



Database



Account Created

Important:

Database me **original password kabhi store nahi hoga.** ❌

Password ko hash karke store kiya jayega.

User enters:

MyPassword123

Database stores:

hashed_password_value

Isse database leak hone par bhi original passwords directly expose nahi hote.

2. Login

User



Email + Password



Backend



Find User



Verify Password Hash



Authentication Successful



Session / Token



Frontend

3. Token kya hai?

Login successful hone ke baad server user ko ek temporary **authentication credential** deta hai.

Simple example:

User → "Main Shivam hoon."

Server → "Prove karo."

User → Email + Password

Server → "Correct."

Server → "Okay, ye credential rakho."



Authentication Token

Ab har protected request ke saath system ko baar-baar password bhejne ki zarurat nahi.

4. Protected API

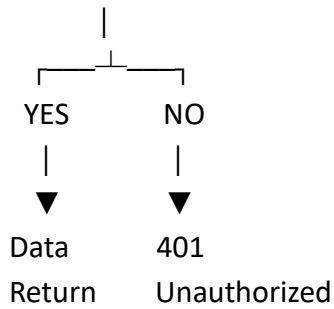
Maan lo participant apni team dekhna chahta hai:

GET /api/v1/my-team

Request ke saath authentication credential jayega.

Backend check karega:

Credential valid?



5. Authentication ke baad Authorization

Sirf valid login enough nahi hai.

Example:

Rahul logged in hai.

Lekin Rahul **Participant** hai.

Woh request karta hai:

DELETE /api/v1/hackathons/123

Backend check karega:

Logged in? ☒

User identified? ☒

Organizer permission? ☐

→ Access Denied

Yani:

Authentication = Tum kaun ho?

Authorization = Tum kya kar sakte ho?

6. Role + Permission Check

Hamare platform me flow kuch aisa hoga:

Request



Authentication



Identify User



Check Organization



Check Role



Check Permission



Allow / Deny

Example:

Organizer A



Edit Hackathon #101



Is #101 owned by Organizer A's organization?



YES → Permission check



Allowed ✅

Agar Hackathon #101 kisi doosri organization ka hai:

Access Denied. 🚫

Golden Rule ★

Never trust the frontend with security decisions.

Frontend button hide kar sakta hai, lekin security nahi de sakta.

Example:

Agar Participant ke frontend me "Delete Hackathon" button nahi dikh raha, iska matlab ye nahi ki woh API manually call nahi kar sakta.

Backend ko har protected request par permission verify karni hi hogi.

Ek important decision

Ab hume decide karna hai ki login ke baad authentication ke liye kya use karenge:

Option A: JWT-based authentication

Option B: Traditional server-side sessions

Option C: Third-party authentication, jaise Google Sign-In

Actually production system me **C ko A/B ke saath combine** bhi kiya ja sakta hai.

Mere according hamare **V1** ke liye simple approach best rahega:

Email + Password + Google Sign-In

Lekin isko 3 parts me samjho.

1. Email + Password — must have

Ye basic authentication hoga.

User:

→ Email enter karega

→ Password enter karega

→ Account create/login karega

Isse platform kisi third-party account par completely dependent nahi rahega.

2. Google Sign-In — optional but useful

Login page par:

Continue with Google

Iska fayda:

- Signup fast
- Password yaad rakhne ki zarurat nahi
- Fake/typo email ka risk kam
- User experience better

Lekin Google Sign-In **authentication ka replacement nahi**, bas login ka additional method hai.

3. JWT ya Session — ye backend implementation detail hai

Abhi tumhe isme confuse hone ki zarurat nahi.

Hum jab actual backend architecture banayenge, tab decide karenge ki **JWT better hai ya server-side session**.

Chapter 31 — Authentication Decision

V1 me platform Email/Password aur Google Sign-In dono support karega.

Email/Password basic authentication method hoga, aur Google Sign-In convenient alternative hoga.

Authentication mechanism (JWT/Session) backend architecture design karte waqt finalize kiya jayega.

Aur ek important correction: **password database me plain text me kabhi store nahi hoga.** Password hashing mandatory hai. 🗝️

Chapter 32: Database ER Diagram

ER Diagram database ke entities aur unke relationships ka visual blueprint hota hai.

ER = **Entity Relationship**

Step 1: Core Entities

Abhi ke liye hamari main entities hain:

User
Organization
Hackathon
Team
Team Member
Submission
Judge
Evaluation
Certificate
Badge
XP
Notification

Lekin ek important baat:

Har entity automatically ek database table nahi hoti.

Kuch entities ko relationships ke liye separate tables banana padega.

Step 2: Sabse pehla Relationship

User → Organization

Ek user organization create kar sakta hai.

Example:

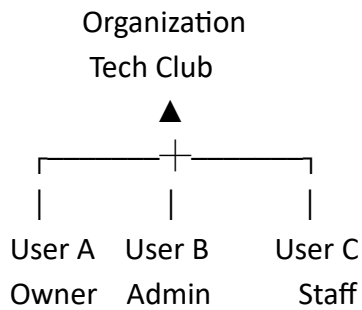
Shivam

|



Tech Club

Lekin ek organization me multiple users kaam kar sakte hain.



Isliye Organization aur User ka relationship simple user_id se solve nahi hoga.

Hume ek **Organization Members** table chahiye.

Step 3: Organization Members

Organization_Members

organization_member_id
organization_id
user_id
organization_role
joined_at
status

Ye table batayegi:

Kaunsa user kis organization ka member hai aur uski organization ke andar kya responsibility hai.

Example:

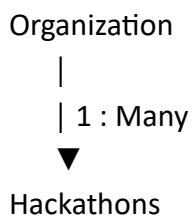
Shivam → Tech Club → Owner

Rahul → Tech Club → Admin

Aman → Tech Club → Staff

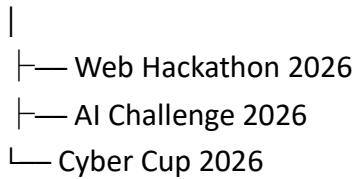
Step 4: Organization → Hackathon

Ek organization multiple hackathons conduct kar sakti hai.



Example:

Tech Club



Isliye Hackathon table me:

organization_id

hona chahiye.

Step 5: Hackathon → Participants

Yahan ek important relationship hai.

Ek participant:

→ Many hackathons join kar sakta hai.

Ek hackathon:

→ Many participants contain kar sakta hai.

Yani:

Many-to-Many Relationship

Iske liye direct participant_id Hackathon table me daalna galat design hoga.

Hume beech me table chahiye:

Hackathon Registrations

Hackathon_Registrations

registration_id

hackathon_id

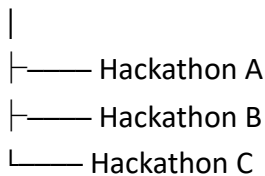
user_id

registered_at

status

Ab:

User



Aur:

Hackathon A

```
|
├── User 1
├── User 2
├── User 3
└── User 4
```

Ye table dono relationships ko manage karegi.

Step 6: Hackathon → Team

Ek hackathon me bahut saari teams ho sakti hain.

Hackathon

```
|
├── Team Alpha
├── Team Beta
├── Team Vision
└── Team CodeX
```

Isliye:

Teams

team_id
hackathon_id
team_name
created_by
status
created_at

Step 7: Team → Users

Yahan bhi **Many-to-Many relationship** hai.

Ek user multiple hackathons me different teams ka member ho sakta hai.

Isliye hume:

Team Members

Team_Members

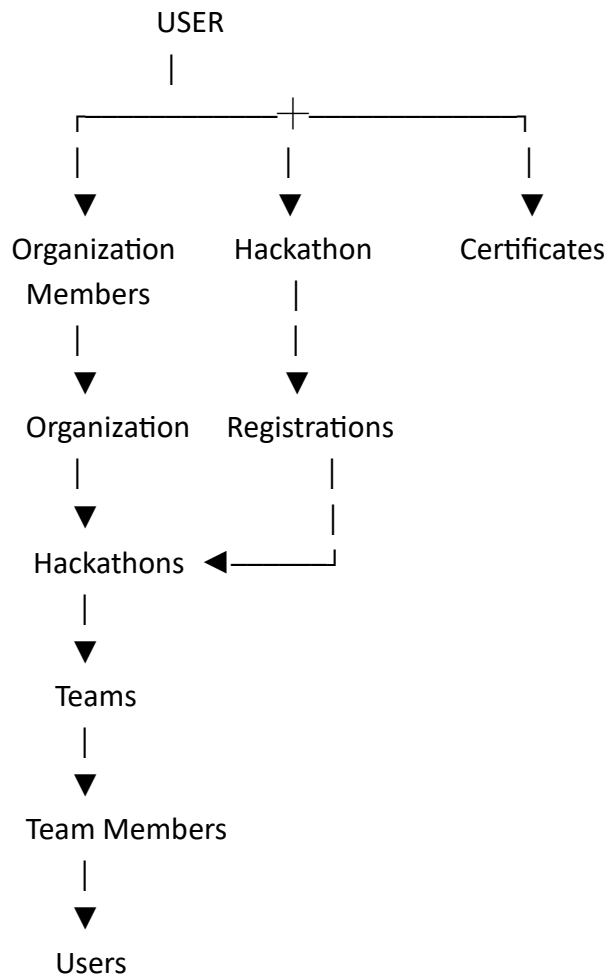
team_member_id
team_id
user_id
role
joined_at
status

Example:

Team Alpha

|
├— Shivam → Leader
├— Rahul → Member
├— Aman → Member
└— Priya → Member

Ab tak ka Blueprint



Abhi diagram **complete nahi hai**.

Next hum isme:

Submission → Judge → Evaluation → Score → Result
add karenge.

★ Ek important concept: Primary Key & Foreign Key

Primary Key (PK) kisi table ke record ki unique identity hoti hai.

Example:

Users

user_id = 101

101 us user ki unique identity hai.

Aur:

Foreign Key (FK) kisi doosri table ke record se relationship banati hai.

Example:

Hackathons

hackathon_id = 501

organization_id = 20

Yahan organization_id batata hai ki Hackathon 501 kis organization ka hai.

Golden Rule 🗝️

Primary Key identifies. Foreign Key connects.

Ye line yaad rakhna. Database design me baar-baar kaam aayegi.

Chapter 33: Submission & Judging Database

A submission aur uski evaluation alag entities hain.

Ek team ek project submit karegi, lekin us project ko multiple judges evaluate kar sakte hain.

1. Submission

Submissions

```
-----
submission_id    PK
team_id          FK
hackathon_id     FK
github_url
demo_url
description
submitted_at
status
```

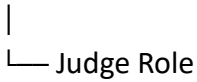
Yahan team_id batata hai ki submission kis team ki hai.

2. Judges

Judge bhi platform ka normal User hi hai.

Isliye alag se judge ka account system nahi banayenge.

User



Lekin kisi particular hackathon me kaun judge hai, ye track karne ke liye:

Hackathon_Judges

hackathon_judge_id PK
hackathon_id FK
user_id FK
status
assigned_at

3. Judge Assignment

Ab ek aur situation:

Ek hackathon me 100 teams aur 5 judges hain.

Ek judge ko specific teams assign karni hain.

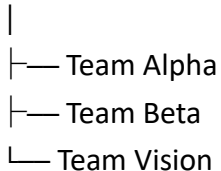
Iske liye:

Judge_Assignments

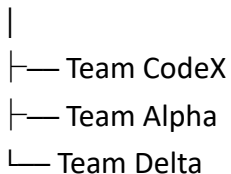
assignment_id PK
hackathon_id FK
judge_id FK
team_id FK
assigned_at
status

Example:

Judge A



Judge B



Notice karo: **Team Alpha ko Judge A aur Judge B dono evaluate kar sakte hain.**

4. Evaluation

Ab judge actual evaluation karega.

Evaluations

evaluation_id PK
assignment_id FK
judge_id FK
submission_id FK
total_score
feedback
submitted_at
status

Yahan ek important rule:

Judge evaluation submit karne ke baad normally usse edit nahi kar sakta.

Agar genuine problem ho, organizer review process ke through correction karwa sakta hai.

5. Evaluation Criteria

Ab humne scoring templates bhi banaye the.

Example:

Innovation	20
Technical	30
UI/UX	15
Problem Solving	20
Presentation	15

Inhe hard-code nahi karenge.

Database me criteria honge:

Evaluation_Criteria

criterion_id PK
hackathon_id FK
name
description
max_score
weight

Example:

Innovation

Max Score = 20

Technical

Max Score = 30

UI/UX

Max Score = 15

6. Actual Scores

Judge ne criteria ke according marks diye.

Evaluation_Scores

score_id PK
evaluation_id FK
criterion_id FK
score

Example:

Evaluation #501

Innovation → 18 / 20

Technical → 27 / 30

UI/UX → 13 / 15

Problem Solving → 17 / 20

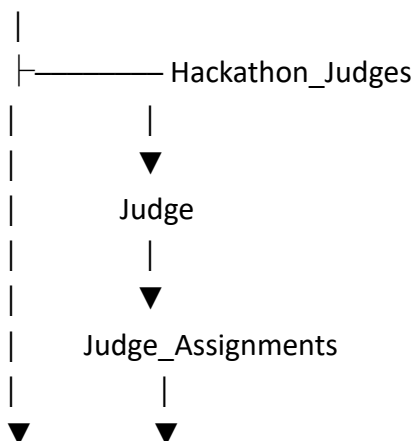
Presentation → 14 / 15

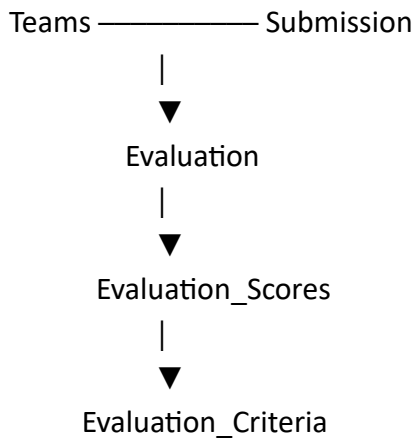
System automatically total calculate karega:

18 + 27 + 13 + 17 + 14 = 89 / 100

Complete Relationship

Hackathon





🔥 Ab ek important architectural decision

Humne pehle decide kiya tha ki **ek project ko minimum 3 judges evaluate karenge**.
Lekin database ko ye assume nahi karna chahiye ki hamesha exactly 3 judges honge.

Organizer kisi hackathon me decide kar sakta hai:

- 1 judge
- 2 judges
- 3 judges
- 5 judges

Platform ko flexible rehna chahiye.

Isliye:

"Minimum 3 judges" hamari recommended judging policy ho sakti hai, database ki hard limitation nahi.
Ye distinction bahut important hai. **Business rule aur database constraint same cheez nahi hote.**

Chapter 34: Hackathon Configuration

Hackathon create karte waqt organizer sirf basic information nahi, balki poore event ke rules configure karega.
Hum ise sections me divide karenge.

1. Basic Information

Hackathon Name

Short Description

Full Description

Banner

Theme

Organization

Example:

TechSprint 2026

Build innovative solutions for real-world problems.

2. Hackathon Type

Organizer choose kar sake:

- Online
- Offline
- Hybrid

Hamare platform ka primary focus online hackathons hai, lekin future ke liye baaki options rakhna smart hai.

3. Registration Settings

Registration Start

Registration End

Maximum Participants (Optional)

Maximum Teams (Optional)

Aur important:

Registration Mode



Individual

OR

Team

OR

Both

Isse different types ke hackathons possible honge.

4. Team Settings

Minimum Team Size

Maximum Team Size

Team Formation Method

Team formation ke options:

Participant creates team

OR

Participant joins existing team

OR

Both

Aur jo humne pehle decide kiya:

Organizer participants ko manually teams me assign nahi karega.

Team formation participant ki choice hogi.

5. Submission Settings

Organizer decide karega ki team kya submit karegi.

Example:

GitHub Repository

Project Description

PPT / PDF

Demo Video

Project ZIP

Live Demo URL

Aur har file ke liye:

Allowed File Type

Maximum File Size

Required / Optional

Example:

GitHub URL → Required

Demo Video → Required

PPT → Optional

ZIP → Optional

6. Judging Settings

Yahan organizer configure karega:

Number of Judges

Judging Start

Judging End

Scoring Criteria

Score Range

Feedback Required?

Example:

Innovation	20
Technical	30
UI/UX	15
Problem Solving	20
Presentation	15

Total:

100

7. Result Settings

Organizer decide kar sakta hai:

Result Publish Date

Leaderboard Visibility

Winner Count

Runner-up Count

Aur ek important setting:

Judges ke individual scores participants ko visible honge ya nahi?

Default me main recommend karunga:

✗ Individual judge scores public nahi.

Participants ko final score aur appropriate feedback mil sakta hai, but judge-by-judge scoring private rahegi unless organizer explicitly chooses otherwise.

8. Hackathon Status

Ye hum pehle discuss kar chuke hain.

Organizer hackathon create karega:

DRAFT



PUBLISHED



REGISTRATION OPEN



REGISTRATION CLOSED



HACKATHON LIVE



SUBMISSION CLOSED



JUDGING



COMPLETED



ARCHIVED

9. Organizer Editable Fields

Aur tumhari earlier requirement ko yahan officially add karte hain.

Organizer publish ke baad change kar sakta hai:

- ✓ Banner
- ✓ Description
- ✓ Team size
- ✓ Some event information
- ✓ Optional links

Normally change nahi kar sakta:

- ✗ Hackathon identity/theme
- ✗ Hackathon type after launch
- ✗ Core rules
- ✗ Important dates after participants have relied on them

Lekin emergency correction ke liye controlled process ho sakta hai.

Golden Rule ★

Not every field should remain editable after publishing.

Kyunki agar organizer registration ke beech me suddenly team size **4 → 8** kar de, to existing participants ke liye unfair situation create ho sakti hai.

🧠 Ab next architectural problem

Maan lo organizer ne hackathon publish kar diya.

Registration chal rahi hai.

Phir organizer bolta hai:

"Mujhe registration deadline 10 August se badhakar 15 August karni hai."

Ye change participants ke liye **fair ho sakta hai**, kyunki unhe extra time mil raha hai.

Lekin doosri taraf:

"Registration deadline 10 August se ghata kar 5 August karni hai."

Ye unfair ho sakta hai, kyunki participants ko expected time se kam mil jayega.

To ab hume **Change Policy** design karni hogi.

Agla chapter isi par hoga:

Chapter 35: Hackathon Change & Version Policy

Publish hone ke baad hackathon ki har information freely editable nahi honi chahiye. Hum changes ko 3 categories me divide karenge.

1. Safe Changes

Aise changes jo competition ki fairness ko affect nahi karte.

Example:

- Banner
- Description correction
- Contact information
- FAQ
- Social links

Organizer directly edit kar sakta hai.

2. Sensitive Changes

Aise changes jo participants ko affect kar sakte hain.

Example:

- Registration deadline
- Submission deadline
- Team size
- Required submission files
- Judging criteria

In changes par system:

Organizer requests change



System checks impact



Change allowed?



YES → Update + Notify participants

NO → Reject / Require special approval

3. Locked Changes

Competition start hone ke baad kuch critical information lock ho jayegi.

Example:

- Hackathon identity
- Core theme
- Fundamental rules
- Judging structure after judging starts

Inhe normal organizer edit nahi kar sakta.

Why changed?

Example:

Organizer: Rahul

Changed: Submission Deadline

Old: 10 Aug, 11:59 PM

New: 12 Aug, 11:59 PM

Reason: Server maintenance

Time: 5 Aug, 4:30 PM

Participant Notification

Agar important change hota hai:

Change



System records change



Participants notified



Updated information visible

Example:

 Submission deadline has been extended from 10 August to 12 August.

Golden Rule

Participants should never be surprised by important rule changes.

Aur ek important concept:

Version History

Hackathon ki information internally versions me maintain ho sakti hai:

Hackathon v1

↓

Hackathon v2

↓

Hackathon v3

Iska matlab ye nahi ki participants ko har version dikhana zaroori hai. System ke paas history rahegi, taaki dispute hone par pata chale ki **kab kya rule tha**.

Chapter 36: Complete Database ER Diagram.

Is chapter ka goal simple hai:

Ab tak jo entities aur relationships humne alag-alag design ki hain, unko ek complete database blueprint me connect karna hai.

Lekin ek warning: **abhi is diagram ko coding-level final schema mat samajhna**. Ye conceptual blueprint hai. Implementation ke time hum fields aur constraints refine karenge.

1. User & Organization

USER

|

└─< ORGANIZATION_MEMBERS >─ ORGANIZATION

|

|

|

└─< HACKATHON

|

└─< HACKATHON_REGISTRATIONS >─ HACKATHON

|

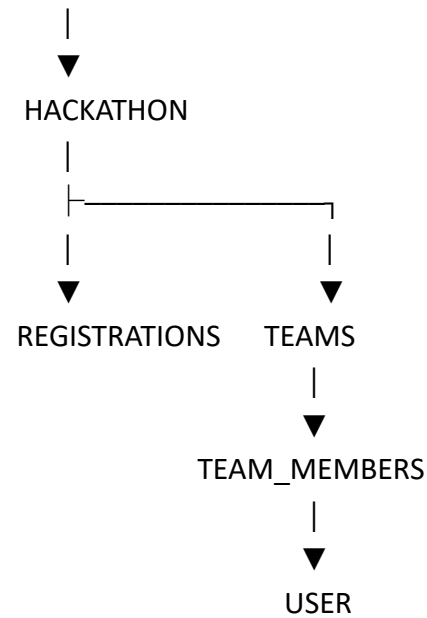
└─< TEAM_MEMBERS >─ TEAM

Iska meaning:

- User multiple organizations me member ho sakta hai.
- Organization multiple hackathons bana sakti hai.
- User multiple hackathons me register kar sakta hai.
- User different hackathons me different teams ka member ho sakta hai.

2. Hackathon & Team

ORGANIZATION



Yahan ek important point:

Team directly user ke saath permanent relation nahi rakhti.

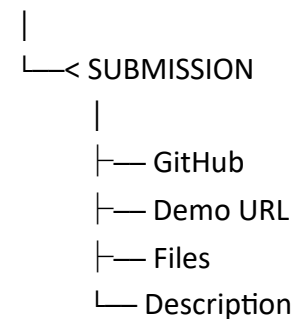
Team membership particular hackathon ke context me hoti hai.

Isliye:

User → Team relationship ko Team_Members table manage karegi.

3. Team → Submission

TEAM



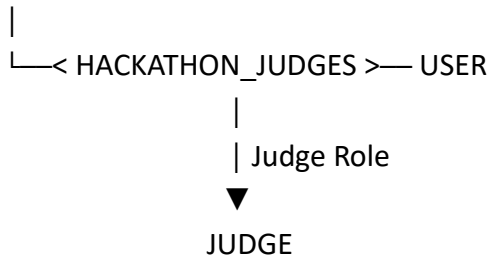
Aur humne decide kiya tha:

Actual files database me nahi, external file storage me rahengi.

Database me sirf file reference/ID rahega.

4. Hackathon → Judges

HACKATHON



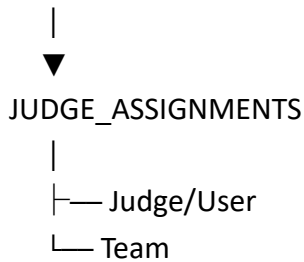
Technically separate JUDGE table mandatory nahi bhi ho sakti, kyunki Judge ek **role** hai.

Ye point important hai:

Judge koi alag type ka human nahi hai; Judge platform user ka role hai.

5. Judge Assignment

HACKATHON



Isse system jaanta hai:

"Is hackathon me is judge ko ye particular team/project evaluate karna hai."

6. Evaluation

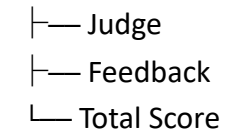
TEAM



SUBMISSION



EVALUATION

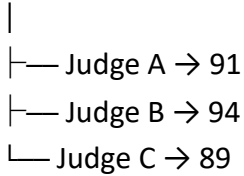


Example:

Team Alpha



Submission #501

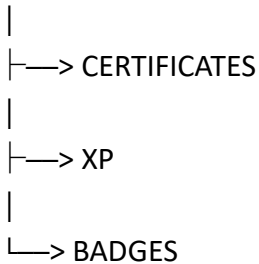


System in scores ke basis par final result calculate karega.

7. Certificates & Gamification

Hackathon complete hone ke baad:

HACKATHON



Lekin yahan bhi ek refinement:

XP aur badges ko hackathon table ke andar directly store nahi karenge.

User ke achievements ko separate system manage karega.

Rule 1:

One user can participate in many hackathons.

Rule 2:

One organization can conduct many hackathons.

Rule 3:

Many-to-many relationships ke liye junction/intermediate tables use karenge.

Examples:

User ↔ Organization

User ↔ Hackathon

User ↔ Team

Judge ↔ Team

Chapter 36 ka final takeaway

Ab hamare paas platform ka **database-level mental model** hai.

Users



Organizations



Hackathons



Teams



Submissions



Judges



Evaluations



Results



Certificates + Achievements

Chapter 37: Frontend Screens & Role-Based Dashboards.

Same platform, different role, different experience.

Participant ko jo dashboard dikhega, Organizer ko wahi dashboard nahi dikhna chahiye.

1. Common Screens

Ye screens almost sabhi users ke liye common hongii:

Landing Page



Hackathon Explore



Hackathon Details



Login / Signup



Profile



Notifications



Settings

2. Participant Dashboard 🧑💻

Participant login karega to usse roughly ye dikhega:

Participant Dashboard

Welcome, Shivam 🙌

XP: 1,250

Level: 5

Badges: 8

Upcoming Hackathons

[Hackathon A]

[Hackathon B]

My Teams

[Team Alpha]

My Submissions

[Project XYZ]

Certificates

[View Certificates]

Notifications

[3 New]

Participant ka main focus:

Discover → Register → Team → Build → Submit → Get Result → Earn Achievement

3. Organizer Dashboard 🏢

Organizer login karega to completely different dashboard:

Organizer Dashboard

Organization: Tech Club

Hackathons

5 Total

2 Draft

1 Live

2 Completed

[Create Hackathon]

Upcoming Events

Registration

Teams

Submissions

Judges
Evaluations
Results
Certificates
Analytics
Organization Settings

Organizer ka main focus:

Create → Configure → Publish → Manage → Judge → Result → Certificate

4. Judge Dashboard 🏛️

Judge Dashboard

Assigned Hackathons

Hackathon A

15 Teams Assigned

8 Evaluated

7 Pending

[Evaluate]

Hackathon B

10 Teams Assigned

10 Evaluated

✓ Completed

Judge ko sirf wahi projects/access dikhna chahiye jo usko assigned hain.

5. Super Admin Dashboard 👑

Super Admin Dashboard

Total Users

Total Organizations

Total Hackathons

Active Hackathons

Reports

User Management

Organization Management

Platform Settings

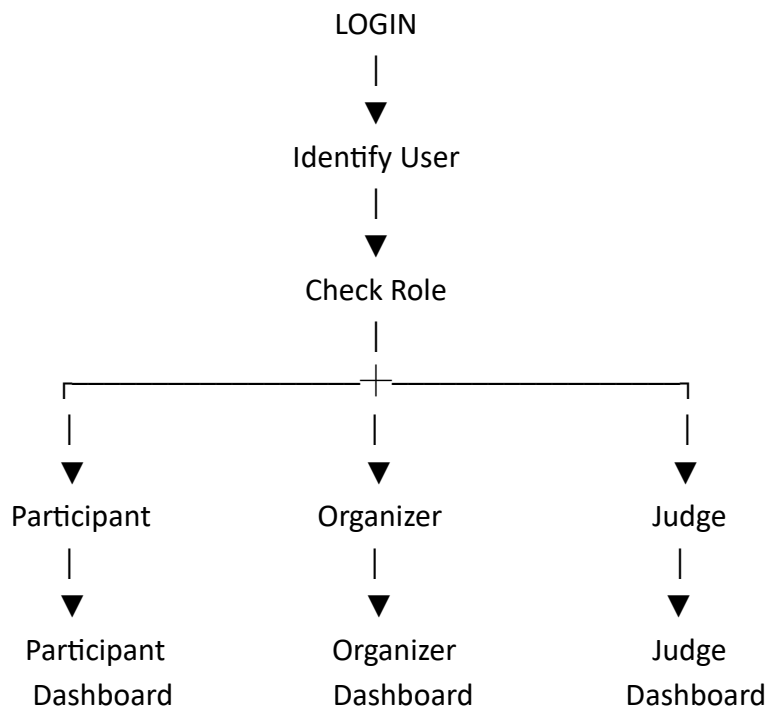
Moderation

Audit Logs

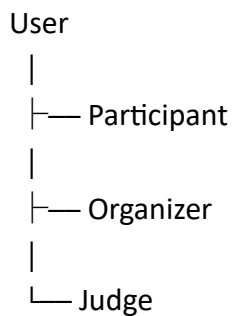
Super Admin ka focus:

Platform health + security + moderation

6. Role-Based Navigation



Agar user ke paas multiple roles hain:



To login ke baad system **role switcher** de sakta hai:

Current Mode:

[Participant ▼]

Participant

Organizer

Judge